

OPTION B INSTITUTE FAMILY

M12 - DISTRIBUTED PROCESSING WITH SPARK / PYSPARK

Standalone Zero-to-Job-Ready Data Engineer 2026

BEGINNER-FIRST MASTER REPAIR RC4

18 canonical lessons | full 16-stage teacher contract | protected attempt-first
review

This learner book is rebuilt from the frozen canonical technical material. It is not a front-matter patch: every lesson is re-laid out into the shared institute teaching grammar while preserving the course competency, source route, guided work, independent proof and validation boundary.

This book is your teacher

Use the book in order. External videos and official docs are visual companions, not hidden prerequisites. Watch or read only the assigned scope, then close it before the independent task.

The 16-stage lesson loop

Why it matters -> What you learn -> Prerequisite -> Picture it -> New words -> Useful source -> Follow once -> Expected result -> Why it worked -> Beginner mistakes -> Recovery -> Independent task -> Validate -> Teach it back -> Protected review + changed retry -> Evidence/mastery + exact next route.

Attempt-first protection

Never open a protected lesson review before saving a genuine independent attempt. If repair is needed, compare only the weak idea, close the review, solve a changed retry, then validate again. A solution seen during review is repair evidence, not mastery.

What counts as mastery

A reproducible artifact or explanation, an independent validation control, preserved failure/recovery evidence, a changed transfer task completed without the worked answer open, and an own-words explanation. Percent-complete is not a substitute.

How to use your devices

Phone: assigned video/reference, reading, revision. Laptop/PC: every real command, query, file, workbook, code task, validation and evidence artifact. Install only what the current lesson asks for.

Contents / Index

This book is your teacher	2
Contents / Index	3
Prerequisite / tool map	4
Practice / review / Gate route	4
DE12-01 - Why Spark exists: driver, executors, partitions and tasks	5
DE12-02 - PySpark setup, SparkSession and laptop-safe local mode	7
DE12-03 - DataFrames and explicit schemas	9
DE12-04 - Spark SQL and expression equivalence	11
DE12-05 - Lazy transformations and actions	13
DE12-06 - Jobs, stages, tasks and physical plans	15
DE12-07 - repartition vs coalesce	17
DE12-08 - Aggregations and shuffle cost	19
DE12-09 - Joins and broadcast strategy	21
DE12-10 - Join correctness, fan-out and reconciliation	23
DE12-11 - Data skew	25
DE12-12 - Adaptive Query Execution (AQE)	27
DE12-13 - Caching and persistence	29
DE12-14 - Parquet I/O and partition pruning	31
DE12-15 - Built-in functions vs Python UDFs	33
DE12-16 - Driver safety: collect, toPandas and bounded inspection	35
DE12-17 - Testing and debugging Spark transformations	37
DE12-18 - Integrated PySpark pipeline and distributed-processing checkpoint	39
Module mastery checklist + revision quick reference	41

Prerequisite / tool map

Use Spark/PySpark safely with explicit schemas, plans, partitions, joins, shuffles and distributed validation.

Item	Rule
Entry prerequisite	Previous module/Gate evidence must be stable.
Primary device	PC/laptop for hands-on work; phone is support only.
External sources	Optional/supporting unless the lesson explicitly marks an official current reference.
Evidence	Module practical + fresh retest + saved evidence; no critical unresolved failure.

Practice / review / Gate route

Lesson teaching -> follow once -> independent attempt -> validate -> save evidence. Only after the genuine attempt may protected review open. If weak: targeted repair -> close review -> changed retry -> revalidate. Finish the module practical/retest route before any downstream Gate that depends on this module.

DE12-01 - Why Spark exists: driver, executors, partitions and tasks

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Explain Spark as a distributed execution engine and distinguish driver, executors, partitions, jobs, stages and tasks.

3. Prerequisite / starting state

Starting state: no prior technical knowledge is assumed beyond the module Start Here instructions.

4. Picture it in your head

Spark exists because one process and one machine eventually become a bottleneck. The driver builds the application plan and coordinates work.

5. New words before we use them

Spark exists because one process and one machine eventually become a bottleneck. The driver builds the application plan and coordinates work. Executors run tasks and hold data/cache for that application.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Sketch 16 orders split across four boxes. Mark one driver that plans a groupBy and four task boxes that process partitions. Then draw a shuffle before the final aggregate. Run the partition/skew simulator. Use it only to visualize uneven buckets; explicitly state that it is not Spark.

8. Expected result / what you should see

Your explanation keeps business row grain separate from physical partitions and says executors execute tasks while the driver coordinates the application.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If everything feels like “a server,” start with one question: who plans, who executes, and what chunk of data does one task receive?

11. If you get stuck

If everything feels like “a server,” start with one question: who plans, who executes, and what chunk of data does one task receive?

12. Small independent task

Draw the driver/executor/partition/job/stage/task relationship from memory and explain what changes when partition count changes.

13. Validate

Your explanation keeps business row grain separate from physical partitions and says executors execute tasks while the driver coordinates the application.

14. Teach it back

Why can changing partition count change performance without changing the correct business answer?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-02

DE12-02 - PySpark setup, SparkSession and laptop-safe local mode

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Install/verify PySpark 4.2.0 safely and create a laptop-safe local SparkSession without treating local mode as a real cluster.

3. Prerequisite / starting state

Prerequisite: DE12-01 mastered with saved evidence.

4. Picture it in your head

PySpark is the Python API for Spark. For this course, first use is here in M12.

5. New words before we use them

PySpark is the Python API for Spark. For this course, first use is here in M12. Spark 4.2.0 supports Python 3.10+ and needs Java 17+.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Setup/orientation; use Teaching Book for laptop-safe setup

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Verify ``java -version``; then ``python -m pip install "pyspark==4.2.0"``; verify ``python -c "import pyspark; print(pyspark.__version__)"``; start a SparkSession with local[2], 2g driver memory and 4 shuffle partitions. Create a session named m12-check, print Spark version/master/shuffle partitions, then stop it cleanly.

8. Expected result / what you should see

Expected: Spark version 4.2.0, local[2], shuffle partitions 4. Exact JVM/OS text may differ.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If Java errors appear, verify Java ≥ 17 and JAVA_HOME before reinstalling Spark. If import fails, check that ``python -m pip`` and ``python`` are the same environment.

11. If you get stuck

If Java errors appear, verify Java ≥ 17 and JAVA_HOME before reinstalling Spark. If import fails, check that ``python -m pip`` and ``python`` are the same environment.

12. Small independent task

Recreate the session from a blank file and explain every builder setting.

13. Validate

Expected: Spark version 4.2.0, local[2], shuffle partitions 4. Exact JVM/OS text may differ.

14. Teach it back

Why do we deliberately use local[2] and 4 shuffle partitions in a teaching lab even though Spark defaults differ?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-03

DE12-03 - DataFrames and explicit schemas

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build DataFrames with explicit schemas and explain why production ingestion should not blindly trust inference.

3. Prerequisite / starting state

Prerequisite: DE12-02 mastered with saved evidence.

4. Picture it in your head

A Spark DataFrame is distributed tabular data with named columns and types plus a query plan. Spark can infer CSV types, but inference costs work and can make contracts depend on sample data.

5. New words before we use them

A Spark DataFrame is distributed tabular data with named columns and types plus a query plan. Spark can infer CSV types, but inference costs work and can make contracts depend on sample data. An explicit StructType makes required types visible and repeatable.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Use the provided order StructType in `m12_reference_pipeline.py` and read the `normal_orders.csv` with `header=true` and `no inferSchema`. `PrintSchema` before any transformation. Create an explicit schema for the retest fixture, then identify which field is a business-validity flag rather than a type rule.

8. Expected result / what you should see

Normal orders: 16 rows, quantity 31, amount 3770.00. `order_id/customer_id/region/category` are strings; `qty` integer; `amount` double.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If numeric columns arrive as strings, check whether an explicit schema was applied and whether malformed data was silently turned null. Inspect bad records rather than casting blindly.

11. If you get stuck

If numeric columns arrive as strings, check whether an explicit schema was applied and whether malformed data was silently turned null. Inspect bad records rather than casting blindly.

12. Small independent task

Close the guided code. From a blank file, define a schema for a fresh six-column shipment fixture, load it without `inferSchema`, and prove row count, types and one business-validity rule independently.

13. Validate

Normal orders: 16 rows, quantity 31, amount 3770.00. order_id/customer_id/region/category are strings; qty integer; amount double.

14. Teach it back

Why is an explicit schema helpful but still insufficient as a full data contract?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-04

DE12-04 - Spark SQL and expression equivalence

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Prove equivalent results through DataFrame expressions and Spark SQL and explain when each style improves maintainability.

3. Prerequisite / starting state

Prerequisite: DE12-03 mastered with saved evidence.

4. Picture it in your head

Spark SQL and the DataFrame API both describe logical transformations that Spark optimizes. Registering a DataFrame as a temp view lets SQL express the same grouped result as ``groupBy().agg()``.

5. New words before we use them

Spark SQL and the DataFrame API both describe logical transformations that Spark optimizes. Registering a DataFrame as a temp view lets SQL express the same grouped result as ``groupBy().agg()``. Equivalence should be proved by result equality, not assumed because both queries look sensible.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

In the reference pipeline, compare the region aggregate built with `groupBy/agg` against the SQL query. Sort both before equality checking because distributed row order is not guaranteed without `ORDER BY`. Write the category total once with DataFrame functions and once with SQL, then compare sorted results.

8. Expected result / what you should see

Both routes must have three category rows and total amount 3770.00.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If equality fails, compare filters, null behavior, aggregation aliases and ordering. Do not blame Spark SQL before comparing logical expressions.

11. If you get stuck

If equality fails, compare filters, null behavior, aggregation aliases and ordering. Do not blame Spark SQL before comparing logical expressions.

12. Small independent task

Close both examples. Rebuild a two-column grouped result once with DataFrame expressions and once with Spark SQL on a changed category/zone grouping, normalize order, and prove exact equality.

13. Validate

Both routes must have three category rows and total amount 3770.00.

14. Teach it back

Why must you sort or otherwise normalize distributed results before using row-order-sensitive equality?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-05

DE12-05 - Lazy transformations and actions

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Predict lazy execution and identify which operations are transformations versus actions.

3. Prerequisite / starting state

Prerequisite: DE12-04 mastered with saved evidence.

4. Picture it in your head

Spark transformations describe new datasets without necessarily executing them immediately. Actions demand a result or side effect and trigger a job.

5. New words before we use them

Spark transformations describe new datasets without necessarily executing them immediately. Actions demand a result or side effect and trigger a job. Lazy planning lets Spark optimize across a chain before execution.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Transformations/actions sections + Teaching Book

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Trace `orders.filter(...).select(...).groupBy(...).agg(...)` and mark where no result is demanded. Then mark `count()` as the action that triggers execution. Write a five-step lineage with exactly one final action and predict when data is actually read.

8. Expected result / what you should see

Your classification must explain why printing a DataFrame object is not the same as materializing all its rows.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you are unsure, ask whether the call returns a new logical DataFrame or asks Spark to compute/return/write a result.

11. If you get stuck

If you are unsure, ask whether the call returns a new logical DataFrame or asks Spark to compute/return/write a result.

12. Small independent task

From memory, write a new lazy lineage with at least four transformations and exactly two actions. Before running it, predict which lines trigger jobs and which only extend the plan; then verify on the learner runtime.

13. Validate

Your classification must explain why printing a DataFrame object is not the same as materializing all its rows.

14. Teach it back

How does lazy execution help optimization, and how can repeated actions surprise a beginner?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-06

DE12-06 - Jobs, stages, tasks and physical plans

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Connect actions to jobs/stages/tasks and use EXPLAIN/UI evidence to reason about the physical plan.

3. Prerequisite / starting state

Prerequisite: DE12-05 mastered with saved evidence.

4. Picture it in your head

A logical query says what result is wanted. Spark produces optimized logical and physical plans describing how to execute it.

5. New words before we use them

A logical query says what result is wanted. Spark produces optimized logical and physical plans describing how to execute it. An action creates one or more jobs; shuffle boundaries often create stage boundaries; tasks execute per partition.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Call `explain(mode="formatted")` on a grouped DataFrame before the action. During a learner-machine run, open the Spark UI and relate the action to job/stage/task counts. Save one plan and annotate Scan, Exchange, Aggregate and any join node you see.

8. Expected result / what you should see

You can explain at least one cause of an Exchange and distinguish plan evidence from a tuning guess.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If the plan is overwhelming, find the data source first, then work upward to filters, joins/exchanges and aggregates.

11. If you get stuck

If the plan is overwhelming, find the data source first, then work upward to filters, joins/exchanges and aggregates.

12. Small independent task

Take a changed aggregation/join pipeline, save formatted EXPLAIN output, annotate scan, filter, exchange, join and aggregate nodes that actually appear, and connect one action to the observed job/stage/task evidence in the Spark UI.

13. Validate

You can explain at least one cause of an Exchange and distinguish plan evidence from a tuning guess.

14. Teach it back

What evidence would convince you that a shuffle happened instead of merely assuming groupBy is expensive?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-07

DE12-07 - repartition vs coalesce

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Choose repartition or coalesce deliberately and explain the shuffle/cost trade-off.

3. Prerequisite / starting state

Prerequisite: DE12-06 mastered with saved evidence.

4. Picture it in your head

``repartition(n)`` can increase or decrease partitions and normally performs a shuffle to redistribute data. ``coalesce(n)`` is commonly used to reduce partitions without a full shuffle, trading balanced distribution for lower movement.

5. New words before we use them

``repartition(n)`` can increase or decrease partitions and normally performs a shuffle to redistribute data. ``coalesce(n)`` is commonly used to reduce partitions without a full shuffle, trading balanced distribution for lower movement. Neither changes the business rows.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Partitions/shuffles context + Teaching Book

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

On the normal DataFrame, record `getNumPartitions()`, then compare `repartition(4)`, `repartition(4,"region")` and `coalesce(1)` plans/partition counts. Write a decision table for increasing parallelism, reducing output files, key distribution and avoiding a single-partition bottleneck.

8. Expected result / what you should see

Every choice includes its intended downstream effect and a correctness control that row count/amount stay 16/3770.00.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you only remember “coalesce is faster,” state whether you are increasing or decreasing partitions and whether balanced redistribution is required.

11. If you get stuck

If you only remember “coalesce is faster,” state whether you are increasing or decreasing partitions and whether balanced redistribution is required.

12. Small independent task

On changed data, choose between repartition and coalesce for two separate goals: balanced keyed work and fewer output files. Record before/after partition counts and defend why neither operation changes business grain.

13. Validate

Every choice includes its intended downstream effect and a correctness control that row count/amount stay 16/3770.00.

14. Teach it back

When is avoiding a shuffle with coalesce a bad trade because the remaining partitions are badly imbalanced?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-08

DE12-08 - Aggregations and shuffle cost

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Measure aggregation shuffle cost and choose partition counts using evidence rather than copying defaults.

3. Prerequisite / starting state

Prerequisite: DE12-07 mastered with saved evidence.

4. Picture it in your head

Aggregations by key require records with the same key to meet, which commonly causes shuffle. Shuffle moves/serializes data and can create many intermediate blocks.

5. New words before we use them

Aggregations by key require records with the same key to meet, which commonly causes shuffle. Shuffle moves/serializes data and can create many intermediate blocks. ``spark.sql.shuffle.partitions`` defaults to 200 in Spark 4.2.0, which is intentionally too many for this tiny course dataset; the lab uses 4 to keep local work visible and light.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Group orders by region and sum amount. Inspect the plan for Exchange. Compare the tiny lab with shuffle partitions 4 versus a deliberately larger value and observe task/partition differences on your machine. Explain why “200 is the Spark default” does not mean “200 is correct for every dataset.”

8. Expected result / what you should see

Region totals must sum back to 3770.00 and grouped row count is four.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If tuning discussion becomes guesswork, return to input size, partition sizes, task count, shuffle bytes and runtime evidence.

11. If you get stuck

If tuning discussion becomes guesswork, return to input size, partition sizes, task count, shuffle bytes and runtime evidence.

12. Small independent task

Build a changed two-key aggregation, reconcile grouped totals back to source totals, then inspect the plan for Exchange/shuffle evidence. Explain why correctness proof must come before performance tuning.

13. Validate

Region totals must sum back to 3770.00 and grouped row count is four.

14. Teach it back

Why can too many tiny shuffle partitions be inefficient even though more partitions sound more parallel?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-09

DE12-09 - Joins and broadcast strategy

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Choose join strategy with size/statistics awareness and use broadcast only when the small side is genuinely safe.

3. Prerequisite / starting state

Prerequisite: DE12-08 mastered with saved evidence.

4. Picture it in your head

Large joins often shuffle data so matching keys meet. If one side is small, Spark can broadcast it to executors and avoid shuffling the larger side.

5. New words before we use them

Large joins often shuffle data so matching keys meet. If one side is small, Spark can broadcast it to executors and avoid shuffling the larger side. Spark 4.2.0 has a 10 MB automatic broadcast threshold by default for suitable plans, but hints are not unconditional guarantees.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Join 16 orders to the 13-row customer dimension with ``broadcast(customers)``. Inspect the formatted plan and look for the broadcast join operator on your learner runtime. Remove the explicit hint and compare the plan. Explain what statistics/threshold information influences automatic broadcast.

8. Expected result / what you should see

Correct join output remains 16 rows, amount 3770.00, and zero unmatched customer segments.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a broadcast hint looks ignored, check join type, plan, table size/statistics and Spark support before forcing another hint.

11. If you get stuck

If a broadcast hint looks ignored, check join type, plan, table size/statistics and Spark support before forcing another hint.

12. Small independent task

Join a changed fact fixture to a deliberately small unique dimension. Prove dimension-key uniqueness, compare an ordinary join with an explicit broadcast-hint plan, and save row/amount controls showing both are semantically identical.

13. Validate

Correct join output remains 16 rows, amount 3770.00, and zero unmatched customer segments.

14. Teach it back

Why should you prove key uniqueness and row-count invariants before celebrating a faster broadcast join?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-10

DE12-10 - Join correctness, fan-out and reconciliation

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Detect fan-out and reconcile joins using row-count, key and measure controls.

3. Prerequisite / starting state

Prerequisite: DE12-09 mastered with saved evidence.

4. Picture it in your head

A join can run successfully and still multiply facts. If a supposed one-side dimension contains duplicate join keys, each matching fact can be duplicated.

5. New words before we use them

A join can run successfully and still multiply facts. If a supposed one-side dimension contains duplicate join keys, each matching fact can be duplicated. DISTINCT after the join hides symptoms and may delete legitimate repeated facts.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Whole lesson + course reconciliation task

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Run `join_fanout_simulator.py`. Compare good and bad joins. Identify the duplicate dimension key and the inflated row/amount controls. In PySpark, load `customers_with_duplicate.csv`, `groupBy(customer_id).count().filter(count>1)`, then prove the join inflates before repairing the dimension contract.

8. Expected result / what you should see

Good join: 16 rows / 3770.00. The bad dimension must produce more rows and a larger amount than the source.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If a join has extra rows, do not add DISTINCT first. Test key uniqueness on each side and locate the key(s) with multiplicity.

11. If you get stuck

If a join has extra rows, do not add DISTINCT first. Test key uniqueness on each side and locate the key(s) with multiplicity.

12. Small independent task

Create a tiny duplicate-key dimension that causes fan-out. Demonstrate the wrong joined row count/measure, identify the duplicated key, repair the dimension contract, rerun, and prove the original fact totals are restored.

13. Validate

Good join: 16 rows / 3770.00. The bad dimension must produce more rows and a larger amount than the source.

14. Teach it back

Why is `DISTINCT` a dangerous response to a fan-out join?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-11

DE12-11 - Data skew

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Measure skew and explain why a few hot keys can dominate stage time.

3. Prerequisite / starting state

Prerequisite: DE12-10 mastered with saved evidence.

4. Picture it in your head

Skew means work is unevenly distributed: one or a few keys/partitions contain much more data than others. A job can appear mostly complete while one straggler task runs much longer.

5. New words before we use them

Skew means work is unevenly distributed: one or a few keys/partitions contain much more data than others. A job can appear mostly complete while one straggler task runs much longer. Measure key-frequency distribution and partition metrics before applying techniques such as salting or AQE skew handling.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Shuffle/skew context + Teaching Book

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Use the simulator region counts as a small visual. Then on Spark `groupBy(region).count()` and inspect max/min counts. The tiny course data is not a performance benchmark; it only teaches the diagnostic pattern. Create a changed in-memory DataFrame with one key repeated far more than others. Predict which partition/task becomes a straggler after hash redistribution.

8. Expected result / what you should see

Your evidence includes key-frequency counts and an explanation of why equal partition count does not guarantee equal partition size.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you cannot see skew, sort key counts descending and compare the largest key with median/typical keys.

11. If you get stuck

If you cannot see skew, sort key counts descending and compare the largest key with median/typical keys.

12. Small independent task

Create a changed skewed-key fixture, calculate key-frequency concentration, predict the straggler key/partition risk, and propose one mitigation only after showing measured skew evidence.

13. Validate

Your evidence includes key-frequency counts and an explanation of why equal partition count does not guarantee equal partition size.

14. Teach it back

How can 100 partitions still perform poorly if one key owns 70% of all records?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-12

DE12-12 - Adaptive Query Execution (AQE)

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Explain AQE and verify which adaptive features are enabled instead of treating it as magic auto-tuning.

3. Prerequisite / starting state

Prerequisite: DE12-11 mastered with saved evidence.

4. Picture it in your head

Adaptive Query Execution uses runtime statistics to revise parts of a SQL physical plan while the query runs. Spark 4.2.0 documents AQE as enabled by default.

5. New words before we use them

Adaptive Query Execution uses runtime statistics to revise parts of a SQL physical plan while the query runs. Spark 4.2.0 documents AQE as enabled by default. It can coalesce post-shuffle partitions, handle skewed shuffle partitions, and sometimes change join strategy.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Execution-plan context + Teaching Book AQE lab

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Print `spark.sql.adaptive.enabled` and inspect a grouped/join plan. On a sufficiently meaningful learner dataset, compare the adaptive/final plan after an action. Write a three-column note: static plan assumption -> runtime fact -> possible AQE adaptation.

8. Expected result / what you should see

Your explanation names at least two AQE behaviors and says correctness controls remain unchanged.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you describe AQE as “Spark automatically makes everything fast,” return to the exact documented adaptive rules and their configuration boundaries.

11. If you get stuck

If you describe AQE as “Spark automatically makes everything fast,” return to the exact documented adaptive rules and their configuration boundaries.

12. Small independent task

On a changed join+aggregate query, record the AQE configuration, capture the initial/adaptive plan evidence available after an action, and state which adaptation Spark may make without claiming AQE fixes incorrect joins.

13. Validate

Your explanation names at least two AQE behaviors and says correctness controls remain unchanged.

14. Teach it back

What kinds of decisions can AQE revise, and what important problems can it not fix for you?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-13

DE12-13 - Caching and persistence

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Use cache/persist only for reused expensive data and prove unpersist lifecycle.

3. Prerequisite / starting state

Prerequisite: DE12-12 mastered with saved evidence.

4. Picture it in your head

Caching stores computed data so later actions can reuse it instead of rebuilding the same lineage. Cache has a memory/storage cost and can become stale relative to a changed external source.

5. New words before we use them

Caching stores computed data so later actions can reuse it instead of rebuilding the same lineage. Cache has a memory/storage cost and can become stale relative to a changed external source. Cache when a reused dataset is expensive enough to justify storage; materialize it with an action, compare repeated-use evidence, and `unpersist()` when done.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Execution/storage context + Teaching Book caching lab

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Create a filtered/derived DataFrame used by two actions. Cache it, materialize with count, inspect storage/UI, run the second action, then unpersist. Write a cache decision for three cases: reused expensive aggregate, one-time filter, tiny dimension broadcast.

8. Expected result / what you should see

You can state what is cached, why it is reused, how it is materialized, and when it is released.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If caching seems to have no benefit, verify the same lineage is actually reused and compare runtime/storage metrics after materialization.

11. If you get stuck

If caching seems to have no benefit, verify the same lineage is actually reused and compare runtime/storage metrics after materialization.

12. Small independent task

Choose one reused expensive lineage and one one-time lineage. Cache/materialize/unpersist only the reused one, leave the one-time lineage uncached, and save evidence supporting both decisions.

13. Validate

You can state what is cached, why it is reused, how it is materialized, and when it is released.

14. Teach it back

Why can caching make a pipeline worse when the dataset is used only once?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-14

DE12-14 - Parquet I/O and partition pruning

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Read/write Parquet with Spark and prove partition pruning without confusing directory partitions with Spark execution partitions.

3. Prerequisite / starting state

Prerequisite: DE12-13 mastered with saved evidence.

4. Picture it in your head

Spark can read and write Parquet while preserving column types and using Parquet metadata. ``partitionBy("region")`` during write creates directory-style dataset partitions.

5. New words before we use them

Spark can read and write Parquet while preserving column types and using Parquet metadata. ``partitionBy("region")`` during write creates directory-style dataset partitions. A filter on a partition column can let Spark prune irrelevant directories.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: DataFrame I/O + Teaching Book Parquet/pruning lab

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

The reference pipeline writes normal orders partitioned by region, then reads back only East and selects `order_id/amount`. Inspect the output directories and formatted plan. Write a second Parquet dataset partitioned by category and compare the files/directories considered for a category filter.

8. Expected result / what you should see

Full read remains 16 rows / 3770.00. East filter returns only East rows. Explain both business and physical partition meanings.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If pruning is absent, verify the filter targets an actual partition column and inspect `PartitionFilters` rather than guessing from runtime alone.

11. If you get stuck

If pruning is absent, verify the filter targets an actual partition column and inspect `PartitionFilters` rather than guessing from runtime alone.

12. Small independent task

Write a changed Parquet dataset partitioned by a different low-cardinality column, read back a filtered subset, reconcile full and filtered controls, and inspect `PartitionFilters`/file-scan evidence for pruning.

13. Validate

Full read remains 16 rows / 3770.00. East filter returns only East rows. Explain both business and physical partition meanings.

14. Teach it back

What is the difference between a Spark execution partition and a Parquet directory partition?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-15

DE12-15 - Built-in functions vs Python UDFs

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Prefer Spark built-in expressions over Python UDFs when practical and explain the optimizer/serialization boundary.

3. Prerequisite / starting state

Prerequisite: DE12-14 mastered with saved evidence.

4. Picture it in your head

Spark SQL built-in functions are visible to Spark's optimizer and execute within Spark's optimized expression engine. A Python UDF crosses a language/runtime boundary and can add serialization overhead while making expression semantics less transparent to Catalyst.

5. New words before we use them

Spark SQL built-in functions are visible to Spark's optimizer and execute within Spark's optimized expression engine. A Python UDF crosses a language/runtime boundary and can add serialization overhead while making expression semantics less transparent to Catalyst. UDFs are still useful when required logic has no suitable built-in; the rule is prefer built-ins, measure before optimizing, and test semantics.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Built-in expression style + Teaching Book UDF comparison

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Create an uppercase/trimmed category with built-ins. Then write an equivalent simple Python UDF only for comparison and inspect plans/runtime on a learner machine. Find three transformations in your pipeline that should stay built-in and one hypothetical case that might justify a UDF.

8. Expected result / what you should see

Built-in and UDF versions must return equivalent values before any performance comparison.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you reach for a UDF first, search current `pyspark.sql.functions` for an equivalent expression and compose built-ins where readable.

11. If you get stuck

If you reach for a UDF first, search current `pyspark.sql.functions` for an equivalent expression and compose built-ins where readable.

12. Small independent task

Without copying the guided example, implement one changed text/conditional transformation using built-in Spark functions and an equivalent Python UDF. Prove equal outputs first, then explain why built-ins remain the default choice.

13. Validate

Built-in and UDF versions must return equivalent values before any performance comparison.

14. Teach it back

Why can a Python UDF be correct yet still be a worse default than a built-in Spark SQL expression?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-16

DE12-16 - Driver safety: collect, toPandas and bounded inspection

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Protect the driver by bounding local collection and distinguishing debugging samples from distributed processing.

3. Prerequisite / starting state

Prerequisite: DE12-15 mastered with saved evidence.

4. Picture it in your head

``collect()`` returns all rows to the driver as Python objects. ``toPandas()`` also materializes data on the driver and should only be used when the result is small enough.

5. New words before we use them

``collect()`` returns all rows to the driver as Python objects. ``toPandas()`` also materializes data on the driver and should only be used when the result is small enough. Large unbounded collection can exhaust driver memory and defeats distributed processing.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Driver vs distributed work context + Teaching Book safety lab

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Use ``orders.limit(5).collect()`` for a tiny debug sample. Compare this with the unsafe habit of calling `collect` on an unknown large source. Rewrite three debugging patterns so each has an explicit bound or aggregated result before reaching the driver.

8. Expected result / what you should see

Every local-inspection action states an expected maximum row count and why it is safe on the learner laptop.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If you need “all rows to inspect,” define the question you are debugging and reduce columns/rows first.

11. If you get stuck

If you need “all rows to inspect,” define the question you are debugging and reduce columns/rows first.

12. Small independent task

Rewrite three unsafe inspection requests into bounded driver-safe checks using `limit/take/aggregates` or `writes`. For each, state the maximum rows expected to reach the driver and why that bound is acceptable.

13. Validate

Every local-inspection action states an expected maximum row count and why it is safe on the learner laptop.

14. Teach it back

Why is ``limit(20).toPandas()`` a different risk profile from ``large_df.toPandas()``?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-17

DE12-17 - Testing and debugging Spark transformations

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Test Spark transformations with deterministic fixtures and debug failures using schema + row + plan evidence.

3. Prerequisite / starting state

Prerequisite: DE12-16 mastered with saved evidence.

4. Picture it in your head

Spark code should be tested as transformations, not only by looking at notebook output. PySpark 4.2 provides testing utilities such as `assertDataFrameEqual` and `assertSchemaEqual`.

5. New words before we use them

Spark code should be tested as transformations, not only by looking at notebook output. PySpark 4.2 provides testing utilities such as `assertDataFrameEqual` and `assertSchemaEqual`. Keep transformation functions small, feed deterministic fixtures, normalize order where order is not part of the contract, and assert schema plus business controls.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Use as visual refresher; testing task remains independent

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

After installing PySpark, run `test_transformations.py`. It builds a tiny `DataFrame` and checks the derived ``net`` column with `assertDataFrameEqual`. Write one test for a join that expects no fan-out and one test for a transformation that deliberately preserves null handling.

8. Expected result / what you should see

Tests are deterministic, order-aware where necessary, and fail when the intended contract is broken.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If equality fails for distributed results, first decide whether row order is part of the contract. Sort by a stable key before comparing when appropriate.

11. If you get stuck

If equality fails for distributed results, first decide whether row order is part of the contract. Sort by a stable key before comparing when appropriate.

12. Small independent task

Write a fresh deterministic unit test for a transformation not used in the bundled example. Include schema/business assertions and a deliberate failing case; save the failure and corrected pass.

13. Validate

Tests are deterministic, order-aware where necessary, and fail when the intended contract is broken.

14. Teach it back

What should a good Spark transformation test prove beyond “the code did not throw an exception”?

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: DE12-18

DE12-18 - Integrated PySpark pipeline and distributed-processing checkpoint

1. Why it matters

This matters because later Distributed Processing with Spark / PySpark work assumes this competency is reliable, not merely familiar. You must be able to reproduce it and validate the result without copying a finished answer.

2. What you are learning

Build and defend an integrated local PySpark pipeline with explicit schema, quality controls, joins, aggregation, plan evidence, safe partitioning, Parquet output and tests.

3. Prerequisite / starting state

Prerequisite: DE12-17 mastered with saved evidence.

4. Picture it in your head

The module checkpoint combines correctness and execution reasoning. Start with explicit schema and source controls.

5. New words before we use them

The module checkpoint combines correctness and execution reasoning. Start with explicit schema and source controls. Transform with built-ins, validate joins before tuning, aggregate and reconcile measures, inspect plans for shuffle/join behavior, write partitioned Parquet, read it back with a pruning filter, and keep driver inspection bounded.

6. Useful video / source

Source: DataTalksClub Zoomcamp — Apache Spark / PySpark

Assigned scope: Review first-look + then build integrated pipeline independently

How to use it: Watch the assigned visual first, then use the Teaching Book for the course-specific build. Close the visual before the independent task.

[Open source](#)

7. Follow with me once

Run the reference pipeline only after PySpark 4.2.0 is available. Then rebuild the same requirements from a blank file rather than copying it. Capture explain output and final controls. Build the changed Clinic Shipments retest only if assigned; for normal mastery, construct your own equivalent pipeline over normal orders with one deliberate validation failure injected and repaired.

8. Expected result / what you should see

Normal integrated controls: 16 order rows, 31 quantity, 3770.00 amount, 16-row customer join, zero unmatched customer segments, four region groups.

9. Why it worked

The worked path succeeds because it begins from the stated starting state, applies the lesson rule deliberately, and checks an observable result before the independent transfer. The validation—not the fact that a command or click completed—is what supports the claim.

10. Common beginner mistakes

If the integrated job fails, isolate in this order: environment/session -> schema/read -> quality -> join cardinality -> transformation -> action/write -> plan/tuning. Repair the smallest failed layer first.

11. If you get stuck

If the integrated job fails, isolate in this order: environment/session -> schema/read -> quality -> join cardinality -> transformation -> action/write -> plan/tuning. Repair the smallest failed layer first.

12. Small independent task

From blank files and the normal fixtures, build the full integrated M12 pipeline independently: explicit schema, validation, safe join, aggregation, plan evidence, partition decision, Parquet write/read, bounded inspection and tests. Do not copy the reference pipeline.

13. Validate

Normal integrated controls: 16 order rows, 31 quantity, 3770.00 amount, 16-row customer join, zero unmatched customer segments, four region groups.

14. Teach it back

In 90 seconds, explain your pipeline from source grain through physical execution and name one correctness failure plus one performance failure that produce different symptoms.

15. Protected review + changed retry

Only after the independent attempt is saved may you open the matching M12 Review/Repair. Close Review before a changed retry.

16. Evidence / mastery + exact next route

Save code/command, source and output controls, one plan/runtime observation when applicable, one error or uncertainty, the repair, and a 60-90 second explain-back. Do not mark mastered because code merely ran. Exact next route: Proceed to the module practical/Gate route after the module mastery checklist.

Module mastery checklist + revision quick reference

- ☐ Every lesson has a saved genuine independent attempt.
- ☐ Every lesson has at least one independent validation/control appropriate to the tool.
- ☐ Protected reviews were opened only after attempts; any weak lesson has a changed retry after review was closed.
- ☐ You can teach back each lesson in your own words and name one realistic failure mode.
- ☐ Required module practical/retest evidence is saved and no critical defect is unresolved.
- ☐ D+1, D+4, D+10 and D+30 revision events are scheduled for weak or high-value competencies.

Revision cadence

D+1: fresh micro-task. D+4: mixed retrieval. D+10: changed/failure case. D+30: interview/project retrieval. Always attempt from memory before reopening a source.

Release boundary

This book is a static learner artifact. It does not prove live tool/cloud execution or a protected Gate pass. Runtime evidence must be recorded separately when the curriculum requires it.